

1 Konzeptionsphase

Ziel der Arbeit ist die Konzeption und Umsetzung einer Cloud-Architektur für eine einfache Unternehmenswebsite. Die Anwendung muss hochverfügbar betrieben werden, weltweit mit geringer Latenz erreichbar sein und ein Backend automatisch, anhand von Zugriffszahlen, skalieren können. Für den Zweck dieses Projekts wird eine einfache .html als Webseite dienen.

Ausgangsidee und Tragfähigkeit. Der zentrale Gedanke ist, die Website in einem Container bereitzustellen. Ein Edge-Dienst soll sich dann um die Verteilung des Traffics kümmern. Für die Wahl von Azure spricht, dass keine Kreditkarte zwingend erforderlich ist und sich moderne, managed Dienste für globales Routing und autoskalierendes Hosting kombinieren lassen. Gegen die Idee könnte sprechen, dass Containerbetrieb für eine rein statische Seite technisch anspruchsvoller als nötig ist. Dennoch ist dieser Ansatz tragfähig, da er als Lernziel dient und später ein echtes Backend ohne grundlegende Plattformänderung ergänzt werden kann.

Konzept und Architektur. Die Zielarchitektur besteht aus fünf Kernbausteinen:

- **GitHub** als Quellcodeverwaltung
- **GitHub Actions** als CI/CD-Pipeline
- **Azure Container Registry (ACR)** als Container-Image-Registry
- **Azure Container Apps (ACA)** in zwei Regionen als Hosting-Layer mit automatischer Skalierung
- **Azure Front Door** als globaler Entry-Point, der Anfragen weltweit entgegennimmt und zur nächstgelegenen Region routet

Damit wird globale Performance erreicht und gleichzeitig die Hochverfügbarkeit durch Multi-Region-Betrieb erhöht. Die Skalierung erfolgt in ACA über konfigurierbare Minimum-/Maximum-Replikas und HTTP-basierte Skalierungsregeln. Bei Lastspitzen werden automatisch zusätzliche Containerinstanzen gestartet, bei geringer Last wieder reduziert. Das beiliegende Architekturdiagramm visualisiert diese Komponenten und deren Interaktion.

Ablauf. Der Ablauf in der Pipeline ist wie folgt: Bei einem Push auf den `main`-Branch baut GitHub Actions ein Docker-Image aus der HTML-Datei, pusht dieses in die ACR und führt `terraform apply` aus, um (a) ACR, (b) Container Apps in zwei Regionen und (c) Azure Front Door inklusive Routing zu erstellen oder zu aktualisieren. Die Website ist anschließend über die Front-Door-URL erreichbar, wobei Front Door die Anfragen global verteilt und ACA die Containerzahl automatisch anpasst.

Methodik/Tooling und Begründung. Terraform wird gewählt, weil es Cloud-Ressourcen deklarativ beschreibt, Änderungen nachvollziehbar macht und die Reproduzierbarkeit der Architektur sicherstellt. GitHub Actions ergänzt dies als automatisierter Delivery-Prozess, der Quellcodeänderungen ohne manuelle Zwischenschritte in ein deploybares Artefakt (Container-Image) und eine konsistente Infrastruktur überführt. Insgesamt erfüllt das Konzept die geforderten Eigenschaften (Hochverfügbarkeit, globale Performance, Autoscaling) und ist gleichzeitig so ausgerichtet, dass ein späteres Backend als zusätzliche Container App mit Front-Door-Routing integriert werden kann.

